

ASTRA INTEL — Presenter Guide

ASTRA INTEL · SCREEN-RECORDED WALKTHROUGH · 12–15 MINUTES

A screen-recorded walkthrough of the console: what to click, in what order, and what to say. Timings are for a **12–15 minute** recording.

Before you record

"Three minutes of preparation prevents the two failures that actually happen in a demo: a cold model and a rate limit."

- Open the app once and run one throwaway query. The first query pays a model warm-up cost; doing it off-camera means your real query looks instant.
- Maximise to 1920x1080. Set browser zoom to 125% so the readouts are legible on a compressed video.
- Close every terminal window. Nothing on camera should be a traceback.
- Have a spare PDF ready on the Desktop for the upload segment.
- Check `http://127.0.0.1:8502/api/health` shows `"status":"ok"`. If it shows 503 the index is empty and the console will look broken.

Segment	Duration	What it proves
1. Opening	30s	The problem being solved
2. Instrument strip	20s	The numbers are real
3. Ask a question	90s	Cited answers
4. Answer integrity	40s	Refusal is a feature
5. Evidence-only mode	30s	Cost and reproducibility
6. Volume by document	20s	Corpus shape
7. Documents tab	60s	Corpus management
8. Briefing	60s	Map-reduce summarisation
9. Upload	90s	Ingest and dedup
10. Reliability	45s	Engineering depth
11. Close	20s	Summary

1. Opening — the problem (30s)

"Defence analysts work from PDF briefings. The problem isn't finding the PDF, it's that you can't ask questions across a whole corpus and trust the answer. ASTRA INTEL is a grounded question-answering system — every claim traces back to a specific page of a specific document."

Point at the sidebar while naming three things:

- **Query** and **Documents** — the only two destinations. Deliberately two.
- **Maintenance** — Rebuild and Clear, parked away from the primary action.
- The line under the brand: *"Every figure is read from the local index. Nothing here is simulated."*

2. The instrument strip (20s)

Walk the four readouts left to right: **Corpus**, **Model**, **Provider**, **Embeddings**.

"These are real values read from the running backend. Groq, openai-gpt-oss-120b, MiniLM embeddings, three documents and 515 passages. The engine indicator in the sidebar shows whether the model is reachable — not merely whether the process is alive."

/api/state.runtime, which carries stable snake_case keys. The **settings** block is display-only and its keys are human labels like "LLM model", so it is never read programmatically. If someone asks how you avoided hardcoding a provider name into the front end, that is the answer.

3. Ask a question — the core (90s)

Click a **suggestion chip**, or type: **What are the three primary mission types of electronic warfare?**

Press **Run Query**. Talk over the thinking state:

"It's two stages. First hybrid retrieval — dense vectors plus BM25, fused with reciprocal rank fusion — pulls candidate passages. Then the model writes an answer using only those passages."

When the answer lands, point at three things in this order:

1. **Status badge** — **Grounded**
2. **Confidence meter** — around 0.69
3. **Evidence (1)** — page number, similarity score, and the excerpt

"That citation is the entire point. If the model can't cite it, the system marks the answer ungrounded rather than letting it through."

Then ask something the corpus cannot answer: **What is the launch date of the next Asterion programme?**

"No supporting passages, so it refuses. It does not invent. That's the behaviour I care about most."

4. Answer Integrity and filtering (40s)

"Every answer in this session is tallied by status."

Click the **Grounded** tile — the query log below filters automatically. Then click **No Context** — it filters to the refused question only.

"These tiles are filters, not just counters. Click to interrogate the session."

Then name what each status means:

Status	Meaning
Grounded	Every sentence resolves to a retrieved passage.
Partial	Supported, but with unsupported claims. Needs review.
No Context	The corpus does not cover it. Refused, not guessed.
Density	Passages divided by pages parsed. A derived figure, not clickable.

5. Evidence-only mode (30s)

Switch the segmented control to **Evidence**. Ask the first question again.

"Same retrieval, no model call. This is what you use when you need the raw passages — for analyst review, or when you want the answer path to be reproducible and free."

not demo the all-documents `/api/briefings` endpoint. It regenerates a summary for every indexed document and measured at 118 seconds in testing. Only the per-row **Brief** button in step 8 is demo-safe.

6. Volume by Document (20s)

"Passages per document. Unmanned aerial vehicle dominates because it's fifty pages against twelve and fifteen."

7. Documents tab — register and sorting (60s)

Click **Documents** in the sidebar.

1. **Sort by Passages** — click the column header
2. **Sort by Document** — alphabetical
3. **Click a row** — selection highlights and the density panel below populates
4. **Expand Extraction Quality** — the heatmap, one square per page

"Each square is a page, brightness is passage count on a single shared scale. A pale cell means the same thing in a small document as in a large one. Click any column header to re-sort."

8. Document briefing (60s)

Click **Brief** on any row. This takes roughly 20 seconds.

"A map-reduce summarisation of that one document — chunks grouped, summarised independently, then merged into one cohesive brief. The footer says which model generated it and which pages it drew from."

The point to land: the panel briefs **the row you clicked**, not the whole corpus.

"It's rate-limited on the free tier, so it's degraded to the structural fallback — which is the designed behaviour." That reads as competence, not failure. Never re-record around it; the graceful degradation is a better story than a clean success.

9. Upload (90s)

Drag a PDF onto the dropzone, or click **browse**. The file summary appears before you commit.

"Preflight happens in the browser against the same limits the server enforces, so an oversized batch fails here with a specific message instead of a bare 400."

Click **Ingest**. The register gains a row and the corpus readouts update.

Re-drop the same file:

"Content-hash deduplication. It's skipped, not re-ingested. Re-uploading a three-hundred-page manual doesn't silently double the corpus."

Then click **Remove** on that row to restore the clean state for the next segment.

10. The reliability work (45s)

This is what separates a demo from an engineering talk.

Open `/api/health` in a new tab:

"Readiness is real. The index is probed, the model client is built and checked. If the index were empty, this returns 503, not a cheerful 200. A health check that can't fail is decoration."

Then point back at the **Maintenance** block:

"Rebuild and Clear require an explicit confirmation token, not just a browser dialog. The dialog stops accidents. The token stops scripts."

Close with the ingestion guards:

"Uploads are capped by both count and size, streamed rather than buffered into memory, and a corrupt PDF comes back with a specific reason instead of a generic failure."

11. Close (20s)

"Two tabs, because two is what an analyst actually does — ask, and manage the corpus. No dashboard nobody opens. Every number on screen is read from the FAISS index at runtime, and every answer is either cited or refused."

Questions you will get

"What if the model makes something up?"

Grounding verification checks each sentence against the retrieved passages. Unsupported numbers trigger a retry, then a refusal. The status badge is never optimistic.

"How is this different from ChatGPT?"

ChatGPT has no access to your corpus. This retrieves from your documents, cites page numbers, and refuses rather than guessing.

"How does it scale?"

Hybrid retrieval — dense plus BM25 fused with reciprocal rank fusion. Document diversity reserves context slots for competitive rivals. The BM25 index is fingerprinted so it only rebuilds when the corpus genuinely changes.

"Where is it deployed?"

Docker, on Railway or Render, with a persistent volume for the index. On first boot it seeds the starter PDFs if the index is empty, and skips the seed when data already exists so uploads survive restarts.

"What about security?"

Be honest: destructive endpoints have no authentication yet, only confirmation tokens. Say auth is next. Do not claim it is protected.

"Why FAISS and not a vector database?"

For a few hundred chunks FAISS is a local file, which means the index survives restarts with no external service, no API key and no network dependency. It also means the whole corpus is inspectable on disk. At a scale where that stops being true, the retriever interface is the seam where a real vector store would go.

"Why did Vercel not work?"

Bundle size — torch alone is around 500 MB against a 500 MB Python function limit. Plus a 4.5 MB request body limit against a 50 MB upload feature, and a read-only filesystem that cannot hold a FAISS index. It needs a writable disk and a long-lived process, so it went to a container platform instead.